

```

1 #=====
2 # MIT 6.046J 代码精读 • PS5 - Choosing Prizes DP
3 # 源文件: choosing_prizes.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS5 Problem 5-2: Choosing Prizes -- four DP variants.
7
8 Given n prizes with nonnegative integer values, pick at most m of them to
9 maximize total value, under different structural constraints. Each function
10 returns (best_value, chosen_indices/prizes).
11
12 (a) plain: pick the m largest values (as a subsequence in original order).
13 (b) two types A,B: all chosen A's must precede all chosen B's in S.
14 (c) non-decreasing: chosen values must form a non-decreasing subsequence.
15 (d) tree: chosen prizes form a connected subtree containing the root.
16
17 A brute-force oracle is provided for each and used in the tests.
18 """
19 from __future__ import annotations
20
21 import itertools
22 from typing import List, Optional, Tuple
23
24
25 # -----
26 # (a) at most m, maximise sum, order preserved
27 # -----
28 def prizes_plain(values: List[int], m: int) -> Tuple[int, List[int]]:
29     """Pick <= m prizes maximising the sum; return (value, chosen indices in
30     original order). Since values are nonnegative, take the m largest.
31     O(n log n)."""
32     order = sorted(range(len(values)), key=lambda i: values[i], reverse=True)
33     chosen = sorted(order[:m])
34     return sum(values[i] for i in chosen), chosen
35
36
37 # -----
38 # (b) two types, all A before all B in the output subsequence
39 # -----
40 def prizes_two_types(values: List[int], types: List[str], m: int) -> Tuple[int, List[int]]:
41     """All type-A picks precede all type-B picks in S (S follows original order).
42
43     Split at some index p (0..n): choose some A-prizes from among indices < p
44     that are type A, and some B-prizes from indices >= p that are type B... but
45     the "A before B in S" with S in original order means: there is a cut position
46     c such that every chosen A has original index < index of every chosen B.
47     Equivalent: pick a boundary rank; A's chosen come from a prefix, B's from the
48     suffix. DP over (prefix boundary, count) is O(n*m); we implement the clean
49     O(n*m) version: for each split point t in 0..n, best = (top A-values among
50     first t that are A) + (top B-values among last n-t that are B), with total
51     count <= m. We precompute prefix/suffix best-k sums.
52     """
53     n = len(values)
54     # bestA[t][j] = max sum of j type-A prizes among indices [0, t)
55     # We only need, for each t, the sorted A-values in prefix; combine with suffix B.
56     # Compute prefixA sorted desc incrementally, suffixB sorted desc incrementally.
57     best = -1
58     best_choice: List[int] = []
59
60     # Precompute, for each boundary t, top A-values in [0, t) and top B-values in [t, n)
61     # prefix_A_vals[t] = list of A values (with idx) in [0, t)
62     prefixA: List[List[Tuple[int, int]]] = [[] for _ in range(n + 1)]
63     for t in range(1, n + 1):
64         prefixA[t] = prefixA[t - 1][:]
65         if types[t - 1] == "A":
66             prefixA[t].append((values[t - 1], t - 1))
67     suffixB: List[List[Tuple[int, int]]] = [[] for _ in range(n + 2)]
68     for t in range(n, -1, -1):
69         suffixB[t] = suffixB[t + 1][:] if t + 1 <= n + 1 else []
70         if t < n and types[t] == "B":
71             suffixB[t].append((values[t], t))
72
73     for t in range(n + 1):
74         A = sorted(prefixA[t], reverse=True)
75         B = sorted(suffixB[t], reverse=True)
76         # choose a of A and b of B, a+b<=m, maximise sum -> greedily merge top values
77         merged = sorted(A + B, reverse=True)[:m]
78         s = sum(v for v, _ in merged)
79         if s > best:
80             best = s
81             best_choice = sorted(i for _, i in merged)
82     return best, best_choice
83
84
85 # -----

```

```

86 # (c) chosen values form a non-decreasing subsequence, length <= m
87 # -----
88 def prizes_nondecreasing(values: List[int], m: int) -> Tuple[int, List[int]]:
89     """Max-sum non-decreasing subsequence of length <= m. DP:
90     dp[i][j] = max sum of a non-decreasing subseq ending exactly at i with j items.
91     O(n^2 m)."""
92     n = len(values)
93     if n == 0 or m == 0:
94         return 0, []
95     NEG = float("-inf")
96     dp = [[NEG] * (m + 1) for _ in range(n)]
97     parent = [[None] * (m + 1) for _ in range(n)]
98     for i in range(n):
99         dp[i][1] = values[i]
100         for k in range(i):
101             if values[k] <= values[i]:
102                 for j in range(2, m + 1):
103                     if dp[k][j - 1] != NEG and dp[k][j - 1] + values[i] > dp[i][j]:
104                         dp[i][j] = dp[k][j - 1] + values[i]
105                         parent[i][j] = (k, j - 1)
106     best = 0
107     end = None
108     for i in range(n):
109         for j in range(1, m + 1):
110             if dp[i][j] > best:
111                 best = dp[i][j]
112                 end = (i, j)
113     chosen: List[int] = []
114     cur = end
115     while cur is not None:
116         i, j = cur
117         chosen.append(i)
118         cur = parent[i][j]
119     chosen.reverse()
120     return best, chosen
121
122 # -----
123 # (d) connected subtree containing the root, at most m nodes
124 # -----
125 def prizes_tree(value: List[int], left: List[int], right: List[int], m: int,
126                 root: int = 0) -> Tuple[int, List[int]]:
127     """Binary tree (left[i], right[i] are child indices or -1). Choose <= m
128     prizes forming a connected subtree rooted at `root` maximising value.
129     Tree knapsack: dp[u][j] = best value of a connected subtree of u's subtree
130     that INCLUDES u and uses exactly j nodes. Combine children by knapsack.
131     O(n * m^2)."""
132     NEG = float("-inf")
133
134     def children(u):
135         cs = []
136         if left[u] != -1:
137             cs.append(left[u])
138         if right[u] != -1:
139             cs.append(right[u])
140         return cs
141
142     # dp[u] : list where dp[u][j] = best value using exactly j nodes of subtree(u),
143     # connected and including u; dp[u][0] = 0 (take nothing -> value 0, allowed).
144     dp: dict = {}
145
146     def solve(u):
147         cs = children(u)
148         # start: using u alone
149         cur = [NEG] * (m + 1)
150         cur[0] = 0 # take nothing in this subtree
151         cur[1] = value[u] # take just u
152         for c in cs:
153             solve(c)
154             child = dp[c]
155             merged = [NEG] * (m + 1)
156             for j in range(m + 1):
157                 if cur[j] == NEG:
158                     continue
159                 for jc in range(0, m + 1 - j):
160                     # child contributes jc nodes; if jc>0 it must include c, and u
161                     # must already be taken (j>=1) to stay connected.
162                     if jc > 0 and j == 0:
163                         continue
164                     cv = child[jc] if jc > 0 else 0
165                     if cv == NEG:
166                         continue
167                     if cur[j] + cv > merged[j + jc]:
168                         merged[j + jc] = cur[j] + cv
169             cur = merged
170         dp[u] = cur

```

```

172     solve(root)
173     best = max(v for v in dp[root] if v != NEG)
174     # reconstruction: re-run picking the argmax (simple: recompute chosen set)
175     chosen = _reconstruct_tree(value, left, right, m, root, dp, best)
176     return best, chosen
177
178
179
180 def _reconstruct_tree(value, left, right, m, root, dp, best):
181     # Greedy reconstruction: find j achieving best at root, then split among children.
182     def children(u):
183         cs = []
184         if left[u] != -1:
185             cs.append(left[u])
186         if right[u] != -1:
187             cs.append(right[u])
188         return cs
189
190     NEG = float("-inf")
191     chosen: List[int] = []
192
193     def rebuild(u, budget, target):
194         # target = dp value we must achieve using exactly `budget` nodes incl. u (if budget>0)
195         if budget == 0:
196             return
197         chosen.append(u)
198         remaining = budget - 1
199         val_needed = target - value[u]
200         cs = children(u)
201         # distribute remaining among children to hit val_needed
202         # brute over compositions (m small)
203         _distribute(cs, 0, remaining, val_needed, {}, dp, rebuild)
204
205     def _distribute(cs, idx, rem, need, alloc, dp, rebuild):
206         if idx == len(cs):
207             if rem == 0 and need == 0:
208                 for c, jc in alloc.items():
209                     if jc > 0:
210                         rebuild(c, jc, dp[c][jc])
211             return True
212             return False
213         c = cs[idx]
214         for jc in range(rem + 1):
215             cv = dp[c][jc] if jc > 0 else 0
216             if cv == NEG:
217                 continue
218             alloc[c] = jc
219             if _distribute(cs, idx + 1, rem - jc, need - cv, alloc, dp, rebuild):
220                 return True
221         alloc.pop(c, None)
222         return False
223
224     # find budget j giving best
225     for j in range(m + 1):
226         if dp[root][j] == best:
227             rebuild(root, j, best)
228             break
229     return sorted(chosen)
230
231
232 # =====
233 # brute-force oracles
234 # =====
235 def brute_plain(values, m):
236     n = len(values)
237     best, bc = 0, []
238     for r in range(min(m, n) + 1):
239         for combo in itertools.combinations(range(n), r):
240             s = sum(values[i] for i in combo)
241             if s > best:
242                 best, bc = s, list(combo)
243     return best, bc
244
245
246 def brute_two_types(values, types, m):
247     n = len(values)
248     best, bc = 0, []
249     for r in range(min(m, n) + 1):
250         for combo in itertools.combinations(range(n), r):
251             # in original order, all A's before all B's
252             seq_types = [types[i] for i in combo]
253             # valid iff no 'A' appears after a 'B'
254             seen_b = False
255             ok = True
256             for tt in seq_types:
257                 if tt == "B":

```

```

258         seen_b = True
259     elif seen_b:
260         ok = False
261         break
262     if ok:
263         s = sum(values[i] for i in combo)
264         if s > best:
265             best, bc = s, list(combo)
266     return best, bc
267
268
269 def brute_nondecreasing(values, m):
270     n = len(values)
271     best, bc = 0, []
272     for r in range(min(m, n) + 1):
273         for combo in itertools.combinations(range(n), r):
274             vals = [values[i] for i in combo]
275             if all(vals[k] <= vals[k + 1] for k in range(len(vals) - 1)):
276                 s = sum(vals)
277                 if s > best:
278                     best, bc = s, list(combo)
279     return best, bc
280
281
282 def brute_tree(value, left, right, m, root=0):
283     n = len(value)
284     # enumerate all connected subtrees containing root of size <= m
285     best = value[root]
286     best_set = {root}
287
288     def children(u):
289         return [c for c in (left[u], right[u]) if c != -1]
290
291     # frontier expansion
292     from itertools import combinations
293     # collect all nodes; a connected subtree rooted at root is any subset S with
294     # root in S and each non-root node's parent in S.
295     parent = [-1] * n
296     for u in range(n):
297         for c in children(u):
298             parent[c] = u
299     for size in range(1, min(m, n) + 1):
300         for combo in combinations(range(n), size):
301             s = set(combo)
302             if root not in s:
303                 continue
304             if all(u == root or parent[u] in s for u in s):
305                 val = sum(value[u] for u in s)
306                 if val > best:
307                     best, best_set = val, s
308     return best, sorted(best_set)
309

```